

ComicCraft – AI Comic Story Creator

using Gemini Models

Sub topic : **Deployment**

Activity 5.2: Testing and Verifying Local Deployment

- **Programming Language:** Python
- **Framework:** FastAPI
- **Template Engine:** Jinja2
- **AI Technology:** Gemini Models
- **Server:** Uvicorn
- **Testing Environment:** Local Computer

Objective:

To test and verify that all ComicCraft features work correctly after local deployment.

Slide 2 — Starting and Checking the Local Server

Running the Application

Start the FastAPI application using Uvicorn.

```
uvicorn main:app --reload
```

Python Health Check

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/health")
async def health_check():
    return {
        "status": "success",
        "message": "ComicCraft is running"
    }
```

Verification

Open the browser and check:

```
http://127.0.0.1:8000
```

Health-check endpoint:

```
http://127.0.0.1:8000/health
```

Expected Result:

```
{
  "status": "success",
  "message": "ComicCraft is running"
}
```

Slide 3 — Testing User Input and Story Generation

Testing the Comic Creation Process

The user enters comic information through the frontend.

```
from fastapi import FastAPI, Form

app = FastAPI()

@app.post("/generate")
async def generate_story(
    theme: str = Form(...),
    characters: str = Form(...)
):
    prompt = f"""
    Create a short comic story.

    Theme: {theme}
    Characters: {characters}
    """

    # Gemini API call can be performed here

    return {
        "theme": theme,
        "characters": characters
    }
```

Testing Steps

1. Open the ComicCraft homepage.
2. Enter a comic theme.

3. Enter character names.
4. Submit the form.
5. Verify that the request reaches FastAPI.
6. Verify that the Gemini-generated response is returned.

Slide 4 — Functional Testing and Error Verification

Important Test Cases

Error Handling Example

```
@app.get("/error-test")
async def error_test():
    try:
        result = "Application is working"
        return {"result": result}

    except Exception as error:
        return {
            "status": "error",
            "message": str(error)
        }
```

Verification:

Errors should be handled without crashing the complete application.

Slide 5 — Milestone 5 Outcome

Activity 5.2: Testing and Verification Completed

Key Achievements:

- Verified that the FastAPI server starts correctly.
- Tested the homepage and application routes.
- Tested user input functionality.
- Verified the comic generation workflow.
- Tested Gemini API communication.
- Checked Jinja2 template rendering.
- Verified generated comic output.
- Tested basic error handling.
- Confirmed that the application works correctly in the local environment.

Testing Workflow

```
Start Server
  ↓
Open ComicCraft
  ↓
Enter Comic Details
  ↓
Submit Request
  ↓
FastAPI Processing
  ↓
Gemini Story Generation
  ↓
Jinja2 Result Page
  ↓
Verify Comic Output
```

Outcome:

The **ComicCraft** application was tested and verified successfully in the **local deployment environment**, ensuring that the major application components work together correctly.